Smart Commerce - Refined 10-Minute Presentation Speech

Prepared for graduate project presentation and defense.

1. Current Problems in the E-Commerce Industry

Good morning respected supervisor and respected members of the examination board.

I am presenting my graduate project, Smart Commerce. The full title is Smart Commerce: An AI-Driven Smart Multi-Vendor E-Commerce Platform with Role-Based Dashboards, Inventory Intelligence, and Demand Forecasting.

I started this project by looking at a common problem in e-commerce. Selling products online is only one part of the work. Behind every online store, there are products, stock, customers, vendors, orders, delivery status, and business decisions.

In many small and medium e-commerce systems, these things are not managed from one place. Stock is often checked manually. Order status is updated late. Vendors may not know which product needs attention. Admins may have to look at different screens or reports to understand what is happening.

Another important problem is demand planning. A store may know what sold in the past, but it may not get a clear idea of what may sell soon. Because of that, restocking decisions become guesswork. Sometimes products become out of stock. Sometimes the business keeps too much stock of slow-moving items.

So the main problem I focused on is this: how can an e-commerce platform support shopping, operations, inventory visibility, and demand planning in one system?

2. My Solution

To solve this problem, I developed Smart Commerce as a full-stack multi-role e-commerce platform.

My goal was not to build only a simple online shop. I wanted to build a system where different users can work from their own dashboards, and where business data can support better decisions.

The system has a customer storefront, a customer dashboard, a vendor dashboard, and an admin dashboard. Admin and super-admin users can access the admin side. Vendors can manage products, inventory, and orders from their own dashboard. Customers can browse products, add items to the cart, place orders, download receipts, and track order progress.

The system also includes product management, order management, stock adjustment, low-stock alerts, dashboard analytics, audit activity, a rule-based chatbot, and a demand forecasting module.

The forecasting part is important, so I want to explain it clearly. My system does not use deep learning. It does not claim to be an advanced machine learning model. It uses a lightweight Python forecasting script. The script reads previous order data, calculates weighted averages, checks trend direction, extracts simple keywords from product names, and gives predicted units for the next 7, 14, and 21 days.

So, my solution combines e-commerce features with practical inventory intelligence and explainable demand forecasting.

3. System Overview

Now let me give a short overview of the system.

On the customer side, the user can visit the homepage, view products, filter by category, search products, sort by price or rating, open product details, add items to the cart, and complete checkout after login.

After an order is placed, the system creates an order record, calculates subtotal, shipping fee, and total amount, and generates an order number. It also downloads a PDF receipt for the customer. The customer can then view order history and track order status through a timeline.

On the vendor side, the vendor can see a dashboard overview, manage products, add a new product, review orders, and adjust inventory stock.

On the admin side, the admin dashboard shows KPIs, revenue trend, low-stock products, recent orders, top products, and audit activity. Admins can also manage customers, vendors, orders, inventory, low-stock products, and demand forecast results.

The backend supports these features through API modules for authentication, products, orders, users, analytics, audit logs, and health checking. The database stores users, roles, products, categories, orders, inventory logs, and audit logs.

4. System Workflow

Moving to the system workflow, I will explain how the main parts work together.

First, the user logs in with email and password. The backend checks the user record, compares the password using bcrypt, verifies that the account is active, and then returns a JWT access token. The frontend stores this token in an HTTP-only session cookie.

After login, the system checks the role. If the user is a customer, the user can access the customer dashboard. If the user is a vendor, the user goes to the vendor dashboard. If the user is an admin or super-admin, the user goes to the admin dashboard.

For the customer workflow, the customer browses products and adds products to the cart. The cart is handled on the frontend using local storage. During checkout, the customer enters delivery information and chooses a payment method such as cash on delivery, card, or bank.

When the customer confirms the order, the frontend sends the order items and shipping address to the backend. The backend checks the products, prepares the order items, calculates subtotal, adds shipping fee, and stores the order with pending status. The checkout page then downloads a PDF receipt and redirects the customer to the order confirmation page.

For order tracking, the system stores tracking events. The order statuses are pending, processing, shipped, delivered, and cancelled. Admins and vendors can update the order status. When the status changes, the backend adds a new tracking event. If the order becomes shipped, the system also creates shipment details such as carrier, tracking number, estimated delivery, and current location.

For inventory workflow, each product has stock and reorder point values. Admins and vendors can view stock levels and adjust stock. When stock is adjusted, the system records the previous stock, new stock, quantity change, reason, and the user who made the change.

For forecasting workflow, the backend collects active products and non-cancelled orders. It groups sales into recent weekly buckets. Then it sends this product history to the Python forecasting script. The script returns predicted units, trend label, keywords, confidence score, and future forecast values. The admin can view these results in chart and table form.

5. System Architecture and Development Lifecycle

From the technical perspective, Smart Commerce follows a client-server architecture.

The frontend is built with Next.js, React, TypeScript, and Tailwind CSS. It contains server-side pages, reusable components, API route handlers, dashboard layouts, product components, cart logic, and session handling.

The backend is built with Node.js, Express, TypeScript, MongoDB, and Mongoose. It is organized into routes, controllers, services, middleware, and models. This structure helped me keep the code modular and easier to maintain.

Security and validation are also included. The backend uses JWT authentication, role authorization middleware, Zod validation, Helmet, CORS, Morgan logging, and rate limiting.

The Python forecasting script is kept inside the backend scripts folder. The backend runs it as a child process and passes product sales history to it. If Python is not available or the script fails, the backend has a fallback forecasting function.

For development and deployment, the project includes Dockerfiles for both frontend and backend, Docker Compose with MongoDB, a Render configuration for the backend, and a Vercel configuration for the frontend.

6. Technology Stack

The main frontend technologies are Next.js 16, React 19, TypeScript, Tailwind CSS, Recharts, Lucide icons, jsPDF, and Sonner notifications.

The backend technologies are Node.js, Express 5, TypeScript, MongoDB, Mongoose, JWT, bcrypt, Zod, Helmet, CORS, Morgan, and express-rate-limit.

For forecasting, I used a Python script with rule-based statistical logic. For charts, I used Recharts. For receipt generation, I used jsPDF. For containerized setup, I used Docker and Docker Compose.

The database is MongoDB. The main models are User, Role, Product, Category, Order, InventoryLog, and AuditLog.

7. Key Features

Now let me highlight the key features of my project.

The first key feature is role-based access control. The backend protects routes using authentication and authorization middleware. Customers, vendors, admins, and super-admins do not get the same access. Each role sees the part of the system that is relevant to that role.

The second feature is the complete customer shopping flow. Customers can browse products, search and filter products, add items to cart, go to checkout, place an order, download a PDF receipt, and track the order.

The third feature is order management. Admins and vendors can view orders and update order status. The system records tracking events, so the customer can see progress from order placed to delivery.

The fourth feature is inventory intelligence. The system shows stock levels, reorder points, low-stock products, out-of-stock products, and suggested reorder quantity. It also supports manual stock adjustment and stores inventory logs.

The fifth feature is dashboard analytics. The admin dashboard shows KPIs, revenue trend, inventory alerts, recent orders, audit activity, and top products. This gives the admin a quick view of platform activity.

The sixth feature is demand forecasting. This is one of the most important parts of my project. The backend prepares weekly sales history from order data. The Python script then calculates a weighted average, checks whether demand is rising, stable, or cooling, extracts simple product keywords, applies a small keyword-based adjustment, and returns future predicted units. It also returns confidence based on data coverage and total sales volume.

The seventh feature is the chatbot. The chatbot is rule-based. It uses a project knowledge database and product-related matching rules. It can answer questions about products, checkout, receipts, order tracking, dashboards, low stock, demand forecast, demo accounts, and setup. It does not use an external AI model.

These features make the system useful for customers, vendors, and administrators at the same time.

8. Business Impact

The business impact of Smart Commerce is practical.

For customers, the system improves shopping by giving a clear product browsing flow, cart, checkout, PDF receipt, and order tracking.

For vendors, the system provides tools to manage products, view orders, and adjust stock. This helps vendors respond faster to order and inventory changes.

For admins, the system gives better control through KPIs, recent orders, low-stock alerts, top products, audit activity, user management, vendor management, and demand forecast results.

The inventory module helps reduce the risk of missing stock problems. The demand forecast module helps identify products that may need attention soon. Because the forecasting method is explainable, an admin can understand the result instead of seeing only a black-box prediction.

Overall, the system reduces manual monitoring and gives each role a focused dashboard for daily work.

9. Conclusion

Finally, let me conclude.

Smart Commerce is a full-stack e-commerce platform with role-based dashboards, product and order management, inventory monitoring, analytics, audit activity, a rule-based chatbot, and demand forecasting.

The system is not just a simple shopping website. It connects customer shopping, vendor operations, admin control, inventory visibility, and forecasting into one platform.

The forecasting module is implemented in a realistic and explainable way. It uses previous order data and a Python script with weighted average, trend detection, keyword extraction, and confidence scoring. So I can clearly explain how the prediction is produced.

Through this project, I learned how to design a complete full-stack system, protect routes by role, structure backend services, manage MongoDB models, connect frontend and backend APIs, and add a practical forecasting component.

Thank you. I am ready to answer any questions.

Presentation Timing Guide

- Current Problems: 1.5 minutes

- My Solution: 1.5 minutes
- System Overview: 1 minute
- System Workflow: 2.5 minutes
- Architecture and Lifecycle: 1 minute
- Technology Stack: 45 seconds
- Key Features: 2 minutes
- Business Impact: 1 minute
- Conclusion: 45 seconds